Politechnika Wrocławska	**Autor:** Aleksandra Cielanga 284539	**Wydział:** Informatyki i telekomunikacji (W4N) **Rok akademicki:** 3
Grafika komputerowa i komunikacja człowiek-komputer		
Data ćwiczenia: 21.10.2026	Temat ćwiczenia laboratoryjnego: **Podstawy OpenGL**	Ocena:
Nr ćwiczenia: 2		Prowadzący: dr inż. arch. Tomasz Zamojski

Spis treści

1 Wstęp **2**

2 Opis zadań opracowanych w ramach laboratorium **2**

2.1 Zadanie 1 – Podstawowe środowisko i zmiana kolorów wierzchołków trójkąta 2
2.1.1 Cel ćwiczenia 2
2.1.2 Wykonane zadanie 2
2.1.3 Prezentacja i omówienie poszczególnych funkcjonalności 2
2.1.4 Efekty pracy 3
2.2 Zadanie 2 – Rysowanie prostokąta 4
2.2.1 Cel ćwiczenia 4
2.2.2 Wykonane zadanie 4
2.2.3 Prezentacja i omówienie poszczególnych funkcjonalności 4
2.2.4 Efekty pracy 5
2.3 Zadanie 3 – Losowość kolorów i deformacje w prostokącie 6
2.3.1 Cel ćwiczenia 6
2.3.2 Wykonane zadanie 6
2.3.3 Prezentacja i omówienie poszczególnych funkcjonalności 6
2.3.4 Efekty pracy 7
2.4 Zadanie 4 – Dywan Sierpińskiego 8
2.4.1 Cel ćwiczenia 8
2.4.2 Wykonane zadanie 8
2.4.3 Prezentacja i omówienie poszczególnych funkcjonalności 8
2.4.4 Efekty pracy 9
2.5 Zadanie 5 – Trójkąt Sierpińskiego 10
2.5.1 Cel ćwiczenia 10
2.5.2 Wykonane zadanie 10
2.5.3 Prezentacja i omówienie poszczególnych funkcjonalności 10
2.5.4 Efekty pracy 12

3 Wnioski **12**

1 Wstęp

Celem zajęć było poznanie podstaw grafiki 2D oraz biblioteki OpenGL. Podczas laboratorium wykorzystano język Python oraz biblioteki `PyOpenGL` i `GLFW`. Przeanalizowano działanie potoku graficznego, rasteryzację, definiowanie wierzchołków i prymitywów oraz liniową interpolację kolorów RGB. Zaimplementowano też obiekty złożone oraz fraktale.

2 Opis zadań opracowanych w ramach laboratorium

2.1 Zadanie 1 – Podstawowe środowisko i zmiana kolorów wierzchołków trójkąta

2.1.1 Cel ćwiczenia

Przygotowanie środowiska pracy, uruchomienie kodu bazowego oraz przypisanie różnych kolorów do poszczególnych wierzchołków trójkąta.

2.1.2 Wykonane zadanie

Napisano skrypt inicjalizujący okno oraz funkcję `update_viewport()`, która utrzymuje proporcje sceny przy zmianie rozmiaru okna w zakresie $[-100.0, 100.0]$. W funkcji renderującej zdefiniowano trójkąt o wierzchołkach w kolorach: czerwonym, zielonym i niebieskim.

2.1.3 Prezentacja i omówienie poszczególnych funkcjonalności

```
#!/usr/bin/env python3
import sys

from glfw.GLFW import *
from OpenGL.GL import *
from OpenGL.GLU import *

def startup():
    update_viewport(None, 400, 400)
    glClearColor(0.5, 0.5, 0.5, 1.0)

def shutdown():
    pass

def render(time):
    glClear(GL_COLOR_BUFFER_BIT)
    glBegin(GL_TRIANGLES)
    glColor3f(0.0, 0.0, 1.0)
    glVertex2f(-50.0, -50.0)
    glColor3f(0.0, 1.0, 0.0)
    glVertex2f(50.0, -50.0)
    glColor3f(1.0, 0.0, 0.0)
    glVertex2f(0.0, 50.0)
    glEnd()
    glFlush()

def update_viewport(window, width, height):
    if width == 0:
        width = 1
    if height == 0:
        height = 1
    aspect_ratio = width / height

    glMatrixMode(GL_PROJECTION)
    glViewport(40, 45, width, height)
    glLoadIdentity()

    if width <= height:
        glOrtho(-100.0, 100.0, -100.0 / aspect_ratio, 100.0 / aspect_ratio,
                1.0, -1.0)

```

```
    else:
        glOrtho(-100.0 * aspect_ratio, 100.0 * aspect_ratio, -100.0, 100.0,
                1.0, -1.0)

    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

def main():
    if not glfwInit():
        sys.exit(-1)

    window = glfwCreateWindow(400, 400, "zad1", None, None)
    if not window:
        glfwTerminate()
        sys.exit(-1)

    glfwMakeContextCurrent(window)
    glfwSetFramebufferSizeCallback(window, update_viewport)
    glfwSwapInterval(1)

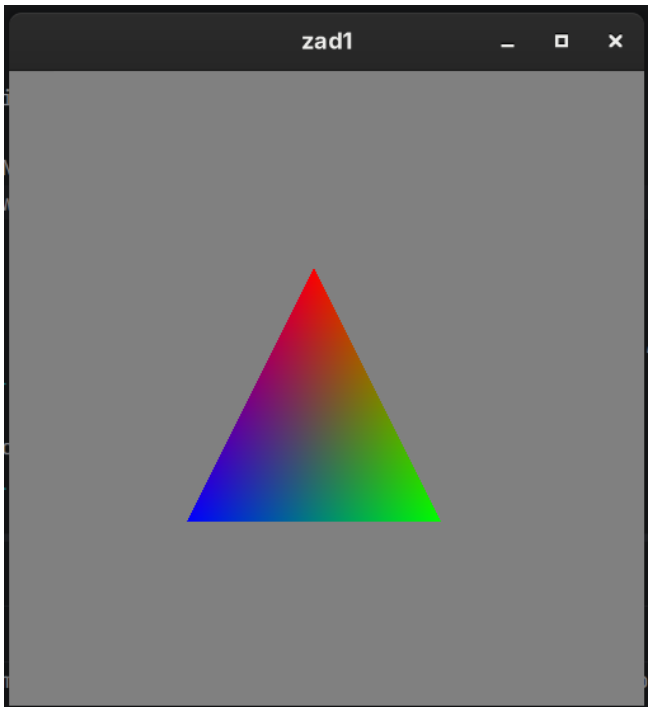
    startup()
    while not glfwWindowShouldClose(window):
        render(glfwGetTime())
        glfwSwapBuffers(window)
        glfwPollEvents()
    shutdown()
    glfwTerminate()

if __name__ == '__main__':
    main()
```

Listing 1: Kod źródłowy dla zadania 1

Wywołanie `glColor3f()` ustala kolor przekazywany do kolejnego wierzchołka `glVertex2f()`. OpenGL interpoluje kolory między wierzchołkami na całej powierzchni trójkąta.

2.1.4 Efekty pracy



2.2 Zadanie 2 – Rysowanie prostokąta

2.2.1 Cel ćwiczenia

Napisanie funkcji rysującej prostokąt o wymiarach a i b w podanym punkcie początkowym x, y z wykorzystaniem dwóch trójkątów.

2.2.2 Wykonane zadanie

Stworzono funkcję `rectangle(x, y, a, b)`, przyjmującą pozycję lewego dolnego rogu oraz długości boków. Na ich podstawie wyznaczono współrzędne pozostałych wierzchołków i utworzono dwa trójkąty połączone wspólną przekątną.

2.2.3 Prezentacja i omówienie poszczególnych funkcjonalności

```
#!/usr/bin/env python3
import sys

from glfw.GLFW import *
from OpenGL.GL import *
from OpenGL.GLU import *

def startup():
    update_viewport(None, 400, 400)
    glClearColor(0.5, 0.5, 0.5, 1.0)

def shutdown():
    pass

def rectangle(x, y, a, b):
    #x,y - placement, a - szerokosc, b - wysokosc
    glBegin(GL_TRIANGLES)

    glColor3f(1, 0, 1)
    glVertex2f(x, y)
    glColor3f(0.5, 0, 1)
    glVertex2f(x + a, y)
    glColor3f(0, 0, 1)
    glVertex2f(x + a, y + b)

    glColor3f(0, 1, 0)
    glVertex2f(x, y)
    glColor3f(0, 1, 0.5)
    glVertex2f(x + a, y + b)
    glColor3f(0, 1, 1)
    glVertex2f(x, y + b)
    glEnd()

def render(time):
    glClear(GL_COLOR_BUFFER_BIT)
    rectangle(-30, -10, 100, 70)
    glFlush()

def update_viewport(window, width, height):
    if width == 0:
        width = 1
    if height == 0:
        height = 1
    aspect_ratio = width / height
    glMatrixMode(GL_PROJECTION)
    glViewport(0, 0, width, height)
    glLoadIdentity()

    if width <= height:
        glOrtho(-100.0, 100.0, -100.0 / aspect_ratio, 100.0 / aspect_ratio, 1.0,
    -1.0)
```

```
    else:
        glOrtho(-100.0 * aspect_ratio, 100.0 * aspect_ratio, -100.0, 100.0, 1.0,
    -1.0)

    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

def main():
    if not glfwInit():
        sys.exit(-1)

    window = glfwCreateWindow(400, 400, "zad2", None, None)
    if not window:
        glfwTerminate()
        sys.exit(-1)
    glfwMakeContextCurrent(window)
    glfwSetFramebufferSizeCallback(window, update_viewport)
    glfwSwapInterval(1)

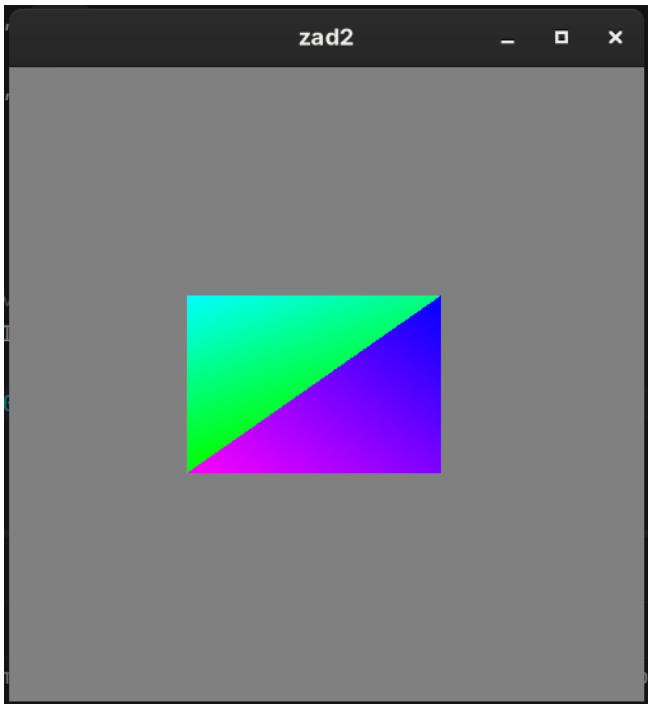
    startup()
    while not glfwWindowShouldClose(window):
        render(glfwGetTime())
        glfwSwapBuffers(window)
        glfwPollEvents()
    shutdown()
    glfwTerminate()

if __name__ == '__main__':
    main()
```

Listing 2: Kod źródłowy dla zadania 2

Podział na dwa trójkąty zapewnia poprawny render figury niezależnie od wsparcia dla złożonych prymitywów w nowszych wersjach sterowników.

2.2.4 Efekty pracy



2.3 Zadanie 3 – Losowość kolorów i deformacje w prostokącie

2.3.1 Cel ćwiczenia

Rozbudowanie funkcji prostokąta o deformacje i wprowadzenie losowych kolorów dla każdego wierzchołka.

2.3.2 Wykonane zadanie

Do funkcji dodano parametr *d*. Do współrzędnych wierzchołków dodano odchylenie `random.uniform(-d, d)`. Aby zapobiec migotaniu kształtu przy każdym odświeżeniu klatki, w funkcji `render()` zastosowano stałe ziarno losowości `random.seed()`.

2.3.3 Prezentacja i omówienie poszczególnych funkcjonalności

```
#!/usr/bin/env python3
import sys
import random
from glfw.GLFW import *
from OpenGL.GL import *
from OpenGL.GLU import *

RANDOMIZE = random.randint(11, 333)
def startup():
    update_viewport(None, 400, 400)
    glClearColor(0.5, 0.5, 0.5, 1.0)

def shutdown():
    pass

def rectangle(x, y, a, b, d=0):
    #x, y - placement, a, b - boki, d - deformacja
    x0 = x + random.uniform(-d, d)
    y0 = y + random.uniform(-d, d)
    x1 = x + a + random.uniform(-d, d)
    y1 = y + random.uniform(-d, d)
    x2 = x + a + random.uniform(-d, d)
    y2 = y + b + random.uniform(-d, d)
    x3 = x + random.uniform(-d, d)
    y3 = y + b + random.uniform(-d, d)

    glBegin(GL_TRIANGLES)
    glColor3f(random.random(), random.random(), random.random())
    glVertex2f(x0, y0)
    glColor3f(random.random(), random.random(), random.random())
    glVertex2f(x1, y1)
    glColor3f(random.random(), random.random(), random.random())
    glVertex2f(x2, y2)

    glColor3f(random.random(), random.random(), random.random())
    glVertex2f(x0, y0)
    glColor3f(random.random(), random.random(), random.random())
    glVertex2f(x2, y2)
    glColor3f(random.random(), random.random(), random.random())
    glVertex2f(x3, y3)
    glEnd()

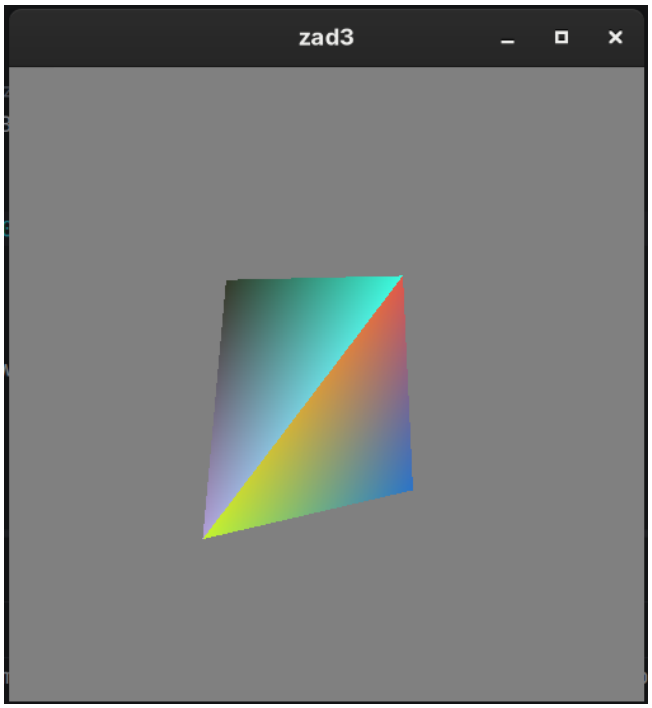
def render(time):
    glClear(GL_COLOR_BUFFER_BIT)
    random.seed(RANDOMIZE)
    rectangle(-20, -30, 80, 100, 15)
    glFlush()

def update_viewport(window, width, height):
    if width == 0:
        width = 1
    if height == 0:
```

```
53          height = 1
54      aspect_ratio = width / height
55      glMatrixMode(GL_PROJECTION)
56      glViewport(0, 0, width, height)
57      glLoadIdentity()
58
59      if width <= height:
60          glOrtho(-100.0, 100.0, -100.0 / aspect_ratio, 100.0 / aspect_ratio, 1.0,
        -1.0)
61      else:
62          glOrtho(-100.0 * aspect_ratio, 100.0 * aspect_ratio, -100.0, 100.0, 1.0,
        -1.0)
63      glMatrixMode(GL_MODELVIEW)
64      glLoadIdentity()
65
66  def main():
67      if not glfwInit():
68          sys.exit(-1)
69      window = glfwCreateWindow(400, 400, "zad3", None, None)
70      if not window:
71          glfwTerminate()
72          sys.exit(-1)
73      glfwMakeContextCurrent(window)
74      glfwSetFramebufferSizeCallback(window, update_viewport)
75      glfwSwapInterval(1)
76
77      startup()
78      while not glfwWindowShouldClose(window):
79          render(glfwGetTime())
80          glfwSwapBuffers(window)
81          glfwPollEvents()
82      shutdown()
83      glfwTerminate()
84
85  if __name__ == '__main__':
86      main()
```

Listing 3: Kod źródłowy dla zadania 3

2.3.4 Efekty pracy



2.4 Zadanie 4 – Dywan Sierpińskiego

2.4.1 Cel ćwiczenia

Zaimplementowanie algorytmu generowania fraktala – dywanu Sierpińskiego – przy pomocy rekurencji.

2.4.2 Wykonane zadanie

Napisano funkcję `carpet(x, y, a, b, depth)`. Prostokąt jest dzielony na siatkę 3×3. Środkowy kwadrat jest pomijany, a dla pozostałych wywoływana jest rekurencja ze zmniejszonymi bokami $a/3$ i $b/3$.

2.4.3 Prezentacja i omówienie poszczególnych funkcjonalności

```
#!/usr/bin/env python3
import sys
from glfw.GLFW import *
from OpenGL.GL import *
from OpenGL.GLU import *

def startup():
    update_viewport(None, 400, 400)
    glClearColor(0.2, 0.2, 0.2, 1.0)

def shutdown():
    pass

def rectangle(x, y, a, b):
    glBegin(GL_TRIANGLES)
    glColor3f(0.4, 0.4, 0)
    glVertex2f(x, y)
    glColor3f(0, 0.8, 0.8)
    glVertex2f(x + a, y)
    glColor3f(0.6, 0.6, 0)
    glVertex2f(x + a, y + b)

    glColor3f(0.5,0,0.5)
    glVertex2f(x, y)
    glColor3f(0, 0.6, 0.6)
    glVertex2f(x + a, y + b)
    glColor3f(0.9, 0.9, 0.9)
    glVertex2f(x, y + b)
    glEnd()

def carpet(x, y, a, b, depth):
    if depth == 0:
        rectangle(x, y, a, b)
    else:
        sub_a = a / 3.0
        sub_b = b / 3.0
        for i in range(3):
            for j in range(3):
                if i == 1 and j == 1: #srodek pusty
                    continue
                carpet(x + i * sub_a, y + j * sub_b, sub_a, sub_b, depth - 1)

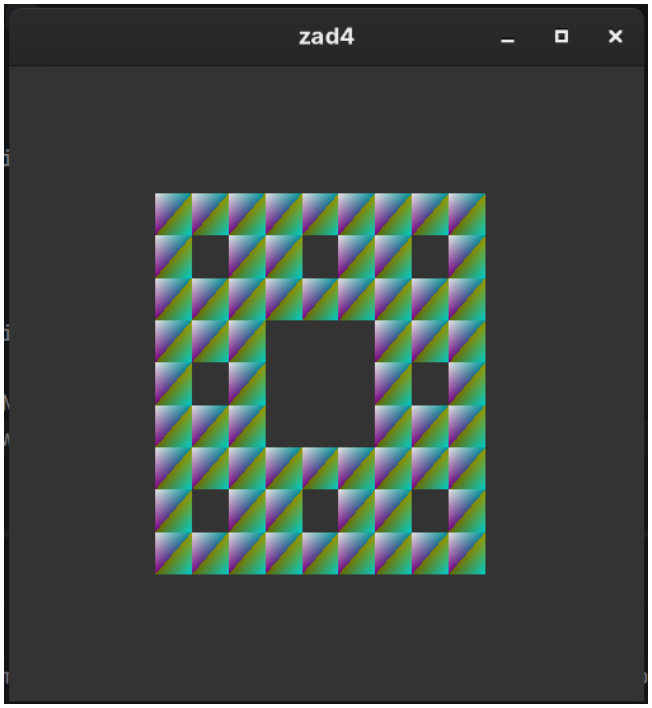
def render(time):
    glClear(GL_COLOR_BUFFER_BIT)
    recursion_depth = 2
    carpet(-70, -70, 130, 150, recursion_depth)
    glFlush()

def update_viewport(window, width, height):
    if width == 0:
        width = 1
    if height == 0:
        height = 1
```

```
54      aspect_ratio = width / height
55      glMatrixMode(GL_PROJECTION)
56      glViewport(55, 40, width, height)
57      glLoadIdentity()
58
59      if width <= height:
60          glOrtho(-100.0, 100.0, -100.0 / aspect_ratio, 100.0 / aspect_ratio, 1.0,
        -1.0)
61      else:
62          glOrtho(-100.0 * aspect_ratio, 100.0 * aspect_ratio, -100.0, 100.0, 1.0,
        -1.0)
63      glMatrixMode(GL_MODELVIEW)
64      glLoadIdentity()
65
66  def main():
67      if not glfwInit():
68          sys.exit(-1)
69      window = glfwCreateWindow(400, 400, "zad4", None, None)
70      if not window:
71          glfwTerminate()
72          sys.exit(-1)
73
74      glfwMakeContextCurrent(window)
75      glfwSetFramebufferSizeCallback(window, update_viewport)
76      glfwSwapInterval(1)
77      startup()
78      while not glfwWindowShouldClose(window):
79          render(glfwGetTime())
80          glfwSwapBuffers(window)
81          glfwPollEvents()
82      shutdown()
83      glfwTerminate()
84
85  if __name__ == '__main__':
86      main()
```

Listing 4: Kod źródłowy dla zadania 4

2.4.4 Efekty pracy



2.5 Zadanie 5 – Trójkąt Sierpińskiego

2.5.1 Cel ćwiczenia

Wygenerowanie trójkąta Sierpińskiego dwiema metodami.

2.5.2 Wykonane zadanie

Zaimplementowano dwa algorytmy:

1. **Metoda rekurencyjna:** Podział trójkąta na mniejsze przez połączenie środków boków.
2. **Gra w chaos:** Wyliczanie punktów w połowie odległości między aktualną pozycją a losowo wybranym wierzchołkiem trójkąta.

Dodano obsługę klawiatury umożliwiającą przełączanie trybów klawiszami `1` i `2`.

2.5.3 Prezentacja i omówienie poszczególnych funkcjonalności

```
#!/usr/init/env python3
import sys
import random
from glfw.GLFW import *
from OpenGL.GL import *
from OpenGL.GLU import *

RANDOMIZE = random.randint(11, 333)
current_mode = 1

TRI_A = (-70, -50)
TRI_B = (70, -50)
TRI_C = (0, 70)

def chaos_points(num_points=10000):
    random.seed(RANDOMIZE)
    px, py = 0, 0
    vertices = [TRI_A, TRI_B, TRI_C]
    points = []
    colors = []

    for _ in range(20):
        v = random.choice(vertices)
        px = (px + v[0]) / 2
        py = (py + v[1]) / 2

    for _ in range(num_points):
        v = random.choice(vertices)
        px = (px + v[0]) / 2
        py = (py + v[1]) / 2
        points.append((px, py))
        colors.append((random.random(), random.random(), random.random()))
    return points, colors

CHAOS_POINTS, CHAOS_COLORS = chaos_points()

def startup():
    update_viewport(None, 400, 400)
    glClearColor(0.5, 0.5, 0.5, 1.0)
    print("Nacisnij 1 - Podzial rekurencyjny")
    print("Nacisnij 2 - Gra w chaos")

def shutdown():
    pass

def key_callback(window, key, scancode, action, mods):
    global current_mode

```

```
    if action == GLFW_PRESS:
        if key == GLFW_KEY_1:
            current_mode = 1
        elif key == GLFW_KEY_2:
            current_mode = 2

def sierpinski_rec(ax, ay, bx, by, cx, cy, depth):
    if depth == 0:
        glBegin(GL_TRIANGLES)
        glColor3f(random.random(), random.random(), random.random())
        glVertex2f(ax, ay)
        glColor3f(random.random(), random.random(), random.random())
        glVertex2f(bx, by)
        glColor3f(random.random(), random.random(), random.random())
        glVertex2f(cx, cy)
        glEnd()
    else:
        abx, aby = (ax + bx) / 2, (ay + by) / 2
        bcx, bcy = (bx + cx) / 2, (by + cy) / 2
        cax, cay = (cx + ax) / 2, (cy + ay) / 2

        sierpinski_rec(ax, ay, abx, aby, cax, cay, depth - 1)
        sierpinski_rec(abx, aby, bx, by, bcx, bcy, depth - 1)
        sierpinski_rec(cax, cay, bcx, bcy, cx, cy, depth - 1)

def chaos_game():
    glPointSize(2.22)
    glBegin(GL_POINTS)
    for (px, py), (r, g, b) in zip(CHAOS_POINTS, CHAOS_COLORS):
        glColor3f(r, g, b)
        glVertex2f(px, py)
    glEnd()


def render(time):
    glClear(GL_COLOR_BUFFER_BIT)

    if current_mode == 1:
        random.seed(RANDOMIZE)
        sierpinski_rec(TRI_A[0], TRI_A[1], TRI_B[0], TRI_B[1], TRI_C[0], TRI_C[1],
    depth=3)
    else:
        chaos_game()
    glFlush()

def update_viewport(window, width, height):
    if width == 0:
        width = 1
    if height == 0:
        height = 1
    aspect_ratio = width / height

    glMatrixMode(GL_PROJECTION)
    glViewport(40, 45, width, height)
    glLoadIdentity()

    if width <= height:
        glOrtho(-100.0, 100.0, -100.0 / aspect_ratio, 100.0 / aspect_ratio,
                1.0, -1.0)
    else:
        glOrtho(-100.0 * aspect_ratio, 100.0 * aspect_ratio, -100.0, 100.0,
                1.0, -1.0)

    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()

```

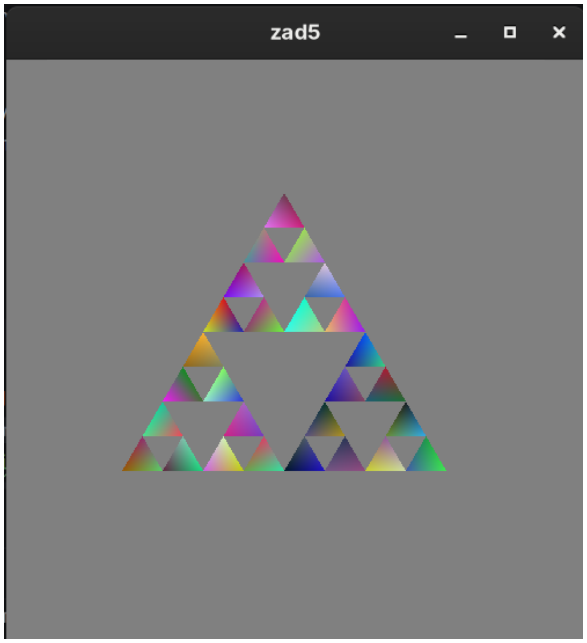
```
def main():
    if not glfwInit():
        sys.exit(-1)
    window = glfwCreateWindow(400, 400, "zad5", None, None)
    if not window:
        glfwTerminate()
        sys.exit(-1)
    glfwMakeContextCurrent(window)
    glfwSetFramebufferSizeCallback(window, update_viewport)
    glfwSetKeyCallback(window, key_callback)
    glfwSwapInterval(1)

    startup()
    while not glfwWindowShouldClose(window):
        render(glfwGetTime())
        glfwSwapBuffers(window)
        glfwPollEvents()
    shutdown()
    glfwTerminate()

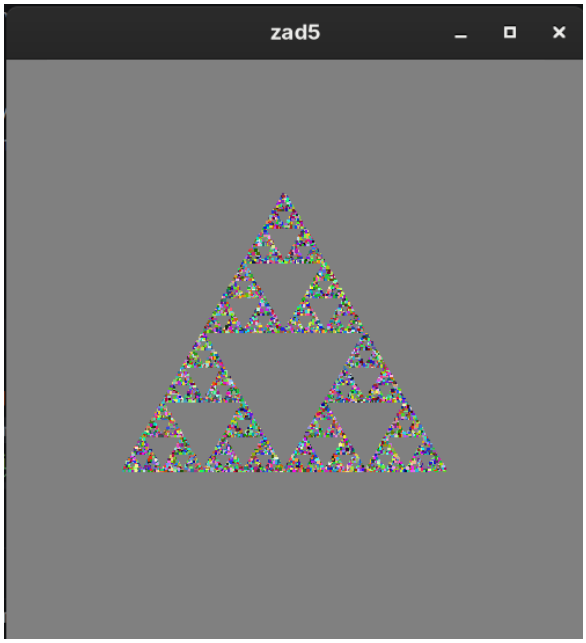
if __name__ == '__main__':
    main()
```

Listing 5: Kod źródłowy dla zadania 5

2.5.4 Efekty pracy



(a) Rekurencyjnie



(b) Chaotycznie

3 Wnioski

OpenGL pozwala na tworzenie oraz wyświetlanie grafiki 2D poprzez bezpośrednie operowanie na wierzchołkach i barwach. Zastąpienie złożonych prymitywów strukturą opartą na trójkątach upraszcza modelowanie geometrii. Wykorzystanie rekurencji oraz metod probabilistycznych pozwala generować fraktale.

Literatura

[1] Sz. Datko, *Lab2: Podstawy OpenGL*, Materiały dydaktyczne, `https://datko.pl/GK/`.

[2] J. de Vries, *LearnOpenGL: Getting Started*, Poradnik programowania grafiki 3D/2D, `https://learnopengl.com/Getting-started/`.

[3] Python Software Foundation, *random — Generate pseudo-random numbers*, Oficjalna dokumentacja języka Python, `https://docs.python.org/3/library/random.html`.